

27 — MULTILAYER PERCEPTRON WITH BACKPROPAGATION (XOR) FROM SCRATCH

Aim

To implement the backpropagation algorithm from scratch and train a multilayer perceptron to learn the non-linearly separable XOR function.

Algorithm + Procedure

1. Define the four input combinations of the XOR function.
2. Define the corresponding target outputs.
3. Initialize the weights randomly for the input-to-hidden and hidden-to-output layers.
4. Apply the sigmoid activation function to the hidden layer.
5. Calculate the output of the neural network using the hidden-layer values.
6. Calculate the output error using the difference between target and predicted output.
7. Calculate the output-layer error using the derivative of the sigmoid function.
8. Propagate the error backward to the hidden layer.
9. Update the weights using gradient descent.
10. Repeat the forward and backward propagation process for several epochs.
11. Calculate the final predicted output.
12. Display the rounded predictions for the XOR inputs.

Code

```
import numpy as np

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_deriv(x):
    return x * (1 - x)

# XOR input
X = np.array([
    [0, 0],
    [0, 1],
```

```

    [1, 0],
    [1, 1]
], dtype=float)
# XOR output
y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=float)
# Set random seed
np.random.seed(1)
# Initialize weights
w1 = np.random.uniform(-1, 1, (2, 4))
w2 = np.random.uniform(-1, 1, (4, 1))
# Learning rate
learning_rate = 0.5
# Training
for epoch in range(10000):
    # Forward propagation
    h = sigmoid(np.dot(X, w1))
    o = sigmoid(np.dot(h, w2))
    # Output error
    err_o = (y - o) * sigmoid_deriv(o)
    # Hidden layer error
    err_h = np.dot(err_o, w2.T) * sigmoid_deriv(h)
    # Update weights
    w2 += np.dot(h.T, err_o) * learning_rate
    w1 += np.dot(X.T, err_h) * learning_rate

```

```
# Final prediction
h = sigmoid(np.dot(X, w1))
o = sigmoid(np.dot(h, w2))
print("Predicted output after training:")
print(np.round(o, 3))
```

Output

```
Predicted output after training:
[[0.032]
 [0.975]
 [0.971]
 [0.028]]
```

Result

The multilayer perceptron was successfully trained using the backpropagation algorithm. The network correctly learned the **XOR function**, which is non-linearly separable.